Due Date

You must *submit* the source code for the solution to this lab exercise to *Moodle* by

Wednesday, October 8, 2025

in order to receive full credit for this work. You must also *demonstrate* the solution to the instructor during class, at the earliest opportunity.

Prime Numbers Lab Exercise

This lab exercise involves enhancing a (partially) working program. The "starter code" finds the first 50 prime numbers and keeps a count of how many divisors were tested in order to determine whether each number is really a prime.

The assignment is to implement another, slightly different, algorithm for finding prime numbers, and keep a count of how many divisors are tested in order to determine whether or not each number is a prime.

The starter code also includes a method named "outputResults" which will display the results of both algorithms, side-by-side on the screen.

Inefficient Prime Number Generator: Sequential Divisors

The method named **`primesSequentialDivisors`** determines whether a number **n** is prime by checking whether **2, 3, 4, 5, 6, ..., n/2** is a divisor with no remainder. If such a divisor is found, then **n** is NOT a prime. (That is, if the integer division **n / divisor** has **remainder == 0**, then **n** is NOT a prime number.)

This code stores each prime number along with the *number of divisor values* which were checked in order to determine that the number is a prime. These are stored in a two-dimensional array for later display. The **divisorCount** variable is really a count of how many times the

```
if (number % divisor == 0)
```

instruction was executed in order to determine that the number is indeed a prime number.

A More Efficient Prime Number Generator: Prime Divisors

A more efficient approach is to check whether any of the prime numbers less than or equal to the square root of **n** can divide **n** evenly (with **remainder == 0**). If not, then **n** is a prime number.

To understand this, consider a few examples:

- There is no point in testing for divisibility by 9 when the program has already checked for divisibility by 3.
- There is no point in testing for divisibility by 15 when the program has already checked for divisibility by 5.

There are actually two advantages which this algorithm has over the first:

- The new algorithm is using ***only*** proven prime numbers as the test divisors, rather than all numbers >= 2 as test divisors. This approach avoids unnecessary passes through the loop.

- The new algorithm also takes advantage of the fact that we do not need to test divisors up to half of the candidate prime number; instead, testing divisors up to the **square root** of the candidate prime number is enough.

Programming Exercise

Implement the code for the **`primesPrimeDivisors`** method, so that it stores the first 50 prime numbers in the array, using the improved approach described above. You will need to store the prime numbers in the array, but use the prime numbers which are *already* in the array as the test divisors of the new "candidate" numbers. Also store in the array the divisor count of how many divisor values must be checked to determine whether each candidate is a prime.

The output for the starter code (`PrimesComparison.java`) is shown on page 3 of this document. This illustrates that the improved algorithm has not been completed, because the two columns on the right side contain only zeroes. The expected output for the completed program is shown on page 4 of this document.

HINT:

- **First**, copy the code from **primesSequentialDivisors** into **primesPrimeDivisors**, and change the copied code so that it uses the array contents as the divisor values.
- **Second**, change the loop continuation condition to check for the **square root of n**, rather than **n/2**. (In the end, the **primesSequentialDivisors** and **primesPrimeDivisors** methods are very similar, but not identical.)

Sample Output

The next two pages of this document contain sample output of the initial ("starter code") program, as well as the expected output from the finished algorithm.

Initial (Starter Code) Output

Prime	SeqDiv	Prime	PrimeDiv
2	0	0	0
3	0	0	0
5	1	0	0
7	2	0	0
11	4	0	0
13	5	0	0
17	7	0	0
19	8	0	0
23	10	0	0
29	13	0	0
31	14	0	0
37	17	0	0
41	19	0	0
43	20	0	0
47	22	0	0
53	25	0	0
59	28	0	0
61	29	0	0
67	32	0	0
71	34	0	0
73	35	0	0
79	38	0	0
83	40	0	0
89	43	0	0
97	47	0	0
101	49	0	0
103	50	0	0
107	52	0	0
109	53	0	0
113	55	0	0
127	62	0	0
131	64	0	0
137	67	0	0
139	68	0	0
149	73	0	0
151	74	0	0
157	77	0	0
163	80	0	0
167	82	0	0
173	85	0	0
179	88	0	0
181	89	0	0
191	94	0	0
193	95	0	0
197	97	0	0
199	98	0	0
211	104	0	0
223	110	0	0
227	112	0	0
229	113	0	0

Desired Result

Prime	SeqDiv	Prime	PrimeDiv
2	0	2	0
3	0	3	0
5	1	5	1
7	2	7	1
11	4	11	2
13	5	13	2
17	7	17	2
19	8	19	2
23	10	23	2
29	13	29	3
31	14	31	3
37	17	37	3
41	19	41	3
43	20	43	3
47	22	47	3
53	25	53	4
59	28	59	4
61	29	61	4
67	32	67	4
71	34	71	4
73	35	73	4
79	38	79	4
83	40	83	4
89	43	89	4
97	47	97	4
101	49	101	4
103	50	103	4
107	52	107	4
109	53	109	4
113	55	113	4
127	62	127	5
131	64	131	5
137	67	137	5
139	68	139	5
149	73	149	5
151	74	151	5
157	77	157	5
163	80	163	5
167	82	167	5
173	85	173	6
179	88	179	6
181	89	181	6
191	94	191	6
193	95	193	6
197	97	197	6
199	98	199	6
211	104	211	6
223	110	223	6
227	112	227	6
229	113	229	6

Copyright © 2025 Peter Morgan. All rights reserved. You may **not** share this document with anyone or use it in any way other than as a participant in this course.